

Nova Options Risk Desk: Industrial Build Plan (v3)

50+ page implementation-spec document tailored to your existing tradingview-recreation codebase

Document date: 2026-02-06 (America/New_York)

Scope: Hackathon submission in ~5–6 weeks + 12■month roadmap. Nova integration is optional and non-blocking.

Author: ChatGPT (GPT■5.2 Thinking)

Document Control

This document is intentionally exhaustive. It is designed to be used as a build manual (requirements + specs + implementation steps + UI/UX + QA + packaging).

Field	Value
Version	v3.0
Audience	You (builder), plus judge-facing packaging guidance
Base repository	/mnt/data/tradingview-recreation-main (2).zip (unzipped locally for analysis)
Non-goals	Not a full broker-grade execution platform; no real-money trading in demo; no vendor market data redis
Key constraint	Nova must be additive; core workflow must run without Nova (future hackathons / provider swaps).
Demo mode	Synthetic snapshot + deterministic tools; zero API keys required to run demo end-to-end.

Reading Guide

- Part 1 audits the current repo (what to keep/refactor) and lists high-impact fixes that improve reliability before you add anything new.
- Part 2 defines the minimal hackathon-winning workflow and the exact acceptance criteria (technical + demo).
- Part 3 is the industrial spec: backend contracts, data models, tools, agents, safety, audit logging.
- Part 4 is UI/UX: information architecture, screen-by-screen layouts, component specs, interactions, empty/loading/error states.
- Part 5 is the execution plan: day-by-day schedule calibrated to your coding velocity and the repo’s existing conventions.
- Part 6 is post-hackathon: 12-month roadmap with epics, milestones, and de-risking steps.

Table of Contents

Document Control	2
Reading Guide	2
Table of Contents	3
Part 1 – Base Repository Audit & Retain/Refactor Map	8
1.1 Quantitative Snapshot of the Current Codebase	8
1.2 Top-Level Repository Structure (What Exists Today)	8
1.3 Backend Structure (phase1/) – Keep vs Refactor vs Deprecate	8
1.4 Frontend Structure (frontend/src/features) – Keep vs Extend	9
Part 1 (continued) – Reliability Fixes Before Adding New Features	11
1.5 Backend Fixes (FastAPI + WebSocket)	11
Fix A: Remove duplicated router includes and make router registration deterministic	11
Fix B: Demo mode must not require Postgres	11
Fix C: Centralize feature flags	11
1.6 Frontend Fixes (Navigation + Stability)	11
Fix D: LeftNavEnhanced.tsx `expanded` variable bug	11
Fix E: Standardize loading/error states for all views	12
1.7 “Platformization” Change (Critical for future hackathons)	12
Part 2 – Hackathon-Winning Scope: Minimal Workflow, Max Credibility	13
2.1 The One Workflow (Non-Negotiable)	13
2.2 Acceptance Criteria (Judge-Proof)	13
2.3 What NOT to Build Before Submission (Explicitly Out of Scope)	13
Part 3 – Product Specification: Options Risk Desk Module	14
3.1 Naming & Placement in Existing Repo	14
3.2 System Context & Data Flow (Text Diagram)	14
3.3 Domain Model (Canonical Objects)	14
3.3.1 Portfolio CSV Canonical Schema	14
3.3.2 Chain Snapshot Schema (Synthetic or BYO Provider)	15
3.3.3 Risk Run & Audit Log	15

Part 3 (continued) – Deterministic Tools, Agents, and Provider-Agnostic AI	17
3.4 Deterministic Tool Suite (Source of Truth for Numbers)	17
Tool T1: portfolio_validator	17
Tool T2: greeks_calculator (primary)	17
Tool T3: stress_tester	17
Tool T4: hedge_generator	17
Tool T5: greeks_verifier (independent)	17
Tool T6: compliance_checker	17
Tool T7: ticket_builder	17
3.5 Agent Orchestration (Keep it Simple)	17
3.6 Provider-Agnostic LLM Layer (Nova is a Plug-in)	18
3.6.1 Interface	18
3.6.2 Router	18
3.6.3 Nova provider implementation (Bedrock)	18
3.7 Nova Act / Automation (Optional, Must Be Safe)	18
Part 4 – UI/UX Specification (Hackathon-Grade + Reusable Platform)	19
4.1 UI Integration Strategy (What to Keep From the Current UI)	19
4.2 Information Architecture (IA)	19
4.3 Risk Desk Screen: Layout Blueprint (Desktop)	19
4.4 Visual Design System (Reuse Existing Theme)	20
Part 4 (continued) – Frontend Component Specifications	21
Component: RiskDeskDashboard	21
Component: RunControls	21
Component: PortfolioUpload	21
Component: PortfolioSummaryTable	22
Component: ValidationBanner	22
Component: DataProvenancePanel	22
Component: ScenarioSelector	23
Component: GreeksCards	23
Component: StressPnLChart	23
Component: HedgeCandidatesTable	24
Component: HedgeDetailPanel	24
Component: ComplianceGatePanel	24
Component: TicketPreviewPanel	25

Component: ToolTracePanel	25
Component: AgentStepper	25
Component: AuditExportPanel	26
Part 4 (continued) – UI Flows, States, and Frontend Engineering Details	27
4.5 Primary Demo Flow (Step-by-Step UI Behavior)	27
4.6 Adversarial / Failure Flow (Must be included once)	27
4.7 Frontend Data Layer & State Management	27
4.8 WebSocket Streaming UI Pattern	27
4.9 Accessibility & Keyboard Support (Lightweight but Real)	28
4.10 UX Copy (Exact Text to Avoid Risky Claims)	28
Part 5 – Backend Specification (Endpoints, Services, Data, and Safety)	29
5.1 API Surface (FastAPI)	29
5.2 Internal Service Interfaces (risk_desk/)	29
5.3 Synthetic Snapshot Generation (Demo Fixture)	29
5.4 Data Provider Adapters (Optional, Feature-Flagged)	29
5.5 Compliance Rules (Deterministic)	30
Part 5 (continued) – What Happens to the AI Options Autopilot	31
5.6 Autopilot + Risk Desk Integration Architecture	31
5.7 Integration Points (Concrete)	31
5.8 Autopilot Modes (Demo vs Future)	31
5.9 Guardrails to Prevent ‘Autopilot as Trading Bot’ Concerns	31
Part 6 – Execution Plan Calibrated to Your Coding Velocity	32
6.1 Velocity Assumptions (Derived From Your Repo)	32
6.2 Branching & Work Discipline (Industrial but Lightweight)	32
6.3 Day-by-Day Plan (42-Day / 6-Week Schedule)	32
Part 7 – Testing, QA, Observability, and Secrets Management	36
7.1 Test Strategy (Minimum for Hackathon + Foundation for Later)	36

7.2 Golden Fixtures and Determinism Checks	36
7.3 Observability (Audit First, Logs Second)	36
7.4 Secrets and Environment Variables	36
7.5 CI Checklist (Light)	36
Part 8 – Submission Packaging (Docs, Commands, and Demo Reliability)	38
8.1 Required Docs to Add/Update	38
8.2 One-Command Demo (Makefile Targets)	38
8.3 Demo Script (3 Minutes, No Branching)	38
8.4 Phrasing That Avoids Risky Claims	38
Part 9 – 12-Month Roadmap (Detailed, Platform-First)	39
9.1 Roadmap Principles	39
9.2 Quarter-by-Quarter Plan	39
9.3 Detailed Epic Breakdown (Month-by-Month)	39
9.4 Reuse for Future Hackathons (Explicit Strategy)	40
Appendix A – Detailed Retention Map (Current Repo → Future State)	41
Appendix A.1 phase1/services/ Directory Inventory	41
Appendix A.2 frontend/src/features/ Inventory	42
Appendix B – Environment Variable Reference (Demo vs Dev vs Prod-like)	44
Appendix C – API Payload Examples (Copy/Paste)	45
C.1 Validate Request/Response	45
C.2 Run Request/Response	45
C.3 Resolve Action Example	45
Appendix D – Final Pre-Submission Checklists	46
D.1 Demo Reliability Checklist	46
D.2 Judge-Facing Checklist	46
D.3 Future Hackathon Reuse Checklist	46

Appendix E – UI Wireframes and Interaction Specifications (Implementation-Level)	47
E.1 Risk Desk Top Bar (RunControls)	47
E.2 Left Column Panels	47
Portfolio Summary Panel	47
Compliance Gate Panel	47
E.3 Center Column Panels	47
Greeks Summary Cards	47
Stress P&L; Chart	47
Hedge Candidates Table	48
E.4 Right Column Panels	48
Tool Trace Panel	48
Audit Export Panel	48
Appendix F – Tool Math, Edge Cases, and Tolerances	49
F.1 Black–Scholes Assumptions (Primary Greeks Tool)	49
F.2 Formula Reference (Used Internally; Not Shown to Users)	49
F.3 Binomial Verifier (American Approximation)	49
F.4 Stress Test Edge Cases	49
F.5 Seeded Scenario Sweep (Optional)	50
Appendix G – Expanded Backlog (Post-Hackathon Task Bank)	51
Data & Providers	51
Risk & Analytics	51
UX	51
Autopilot	51
Infra	51

Part 1 – Base Repository Audit & Retain/Refactor Map

This section is tailored to your uploaded repo. The goal is to preserve your existing momentum (you built fast by experimenting), but add structure so new hackathon features don't collapse the codebase.

1.1 Quantitative Snapshot of the Current Codebase

Approximate line counts (from the zipped repo contents; excludes node_modules and build outputs):

Bucket	Lines (approx)	Notes
Application code (Python/TS/TSX/JS)	111,128	Backend + frontend logic
Tests (Python/TS/TSX/JS)	26,698	pytest + vitest coverage scaffolding
Docs/config (MD/JSON/YAML)	28,229	Large doc footprint; useful for packaging
Other	71,036	Assets, lockfiles, etc.

Implication: you already have a platform. The hackathon feature should be a modular slice that reuses: (a) existing layout/navigation, (b) existing portfolio/options primitives, (c) existing autopilot + LLM router infrastructure, and (d) existing WebSocket streaming patterns.

1.2 Top-Level Repository Structure (What Exists Today)

```
Top-level directories:
- artifacts/
- browser_extension/
- cboe_pipeline/
- data/
- docs/
- frontend/
- n8n/
- phase1/
- qc/
- repo_cleanup_evidence/
- scripts/
- strategies/
- tests/
- tools/
```

Key observation: the repository already contains substantial documentation and multiple subprojects (phase1 backend, frontend, and additional artifacts). Do not restructure everything; instead, introduce a new 'risk desk' module in the existing conventions.

1.3 Backend Structure (phase1/) – Keep vs Refactor vs Deprecate

The backend entrypoint is `phase1/services/api/main.py` (FastAPI) with many routers already included. Your hackathon feature should add a dedicated router namespace and keep demo mode fully local.

Service module	Keep?	Action	Why / Notes
alerts	Keep	Review	Not assessed; keep but review integration points
api	Keep	Refactor	Router hygiene, remove duplicates, add /risk-desk endpoints + WS
automation	Keep	Review	Not assessed; keep but review integration points

autopilot	Keep	Integrate	Autopilot becomes a consumer of risk desk gates; keep strategy logic, add
backtest	Keep	Review	Not assessed; keep but review integration points
bar_engine	Keep	Review	Not assessed; keep but review integration points
charting	Keep	As-is	Same as above
clock	Keep	Review	Not assessed; keep but review integration points
delivery	Keep	Review	Not assessed; keep but review integration points
execution	Keep	Review	Not assessed; keep but review integration points
forecasting	Keep	Review	Not assessed; keep but review integration points
fundamentals	Keep	Review	Not assessed; keep but review integration points
incidents	Keep	Review	Not assessed; keep but review integration points
ingestion	Keep	Review	Not assessed; keep but review integration points
llm	Keep	Extend	Add Bedrock/Nova provider without forcing Nova
monitoring	Keep	Review	Not assessed; keep but review integration points
options	Keep	Extend	Add portfolio/greeks/pnl calculators or reuse if present
packages	Keep	Review	Not assessed; keep but review integration points
parity	Keep	Review	Not assessed; keep but review integration points
persistence	Keep	Review	Not assessed; keep but review integration points
portfolio	Keep	Refactor	Unify schemas; ensure options legs can be expressed cleanly
recovery	Keep	Review	Not assessed; keep but review integration points
replay	Keep	Extend	Deterministic snapshots; risk desk piggybacks replay infra
reports	Keep	Review	Not assessed; keep but review integration points
strategy	Keep	Review	Not assessed; keep but review integration points
tts	Keep	Review	Not assessed; keep but review integration points
verifier	Keep	Review	Not assessed; keep but review integration points

1.4 Frontend Structure (frontend/src/features) – Keep vs Extend

Your frontend already has an opinionated shell + multiple views. The hackathon feature should extend the existing Options UI rather than introduce a parallel layout system.

Feature	Keep?	Action	Notes
alerts	Keep	Review	Keep; only touch if needed for Risk Desk integration
audit	Keep	Review	Keep; only touch if needed for Risk Desk integration
autopilot	Keep	Integrate	Autopilot should consume compliance gate outputs
backtest	Keep	Optional	Use for scenario comparisons; avoid full build in hackathon

chart	Keep	Review	Keep; only touch if needed for Risk Desk integration
dashboard	Keep	As-is	Add risk summary tile later
drawings	Keep	Review	Keep; only touch if needed for Risk Desk integration
fundamentals	Keep	Review	Keep; only touch if needed for Risk Desk integration
health	Keep	Review	Keep; only touch if needed for Risk Desk integration
ide	Keep	Review	Keep; only touch if needed for Risk Desk integration
incidents	Keep	Review	Keep; only touch if needed for Risk Desk integration
indicators	Keep	Review	Keep; only touch if needed for Risk Desk integration
layout	Keep	Extend	Add Risk Desk view/tab; ensure nav remains coherent
notes	Keep	Review	Keep; only touch if needed for Risk Desk integration
observability	Keep	Review	Keep; only touch if needed for Risk Desk integration
options	Keep	Extend	Add Risk Desk tab + new panels
orders	Keep	Review	Keep; only touch if needed for Risk Desk integration
packages	Keep	Review	Keep; only touch if needed for Risk Desk integration
portfolio	Keep	Review	Keep; only touch if needed for Risk Desk integration
replay	Keep	Review	Keep; only touch if needed for Risk Desk integration
reports	Keep	Review	Keep; only touch if needed for Risk Desk integration
runs	Keep	Review	Keep; only touch if needed for Risk Desk integration
strategy	Keep	Review	Keep; only touch if needed for Risk Desk integration
trades	Keep	Review	Keep; only touch if needed for Risk Desk integration
trading	Keep	Review	Keep; only touch if needed for Risk Desk integration
tts	Keep	Review	Keep; only touch if needed for Risk Desk integration

Part 1 (continued) – Reliability Fixes Before Adding New Features

These are the highest ROI changes that directly increase demo reliability and reduce debugging time. They also reduce the chance you regress existing functionality while adding the Risk Desk.

1.5 Backend Fixes (FastAPI + WebSocket)

Fix A: Remove duplicated router includes and make router registration deterministic

In `phase1/services/api/main.py`, some routers appear to be included multiple times (e.g., `autopilot_ws_router` is included twice). In production, duplicates lead to confusing behavior (handlers firing twice, unexpected route collisions).

- Create a single `routers/__init__.py` registry list and iterate once.
- Add a startup assertion: no duplicate (method,path) pairs in OpenAPI routes.
- Add a health endpoint that lists enabled feature flags and active data/LLM providers.

Fix B: Demo mode must not require Postgres

Your `.env` template includes Postgres; keep DB support, but the hackathon demo must run without it. Demo mode should persist runs/audits in memory (or SQLite file) and optionally export JSON.

- Introduce `RUN_MODE=demo|dev|prod` and gate DB init accordingly.
- Ensure API returns useful errors when DB disabled (no stack traces to UI).

Fix C: Centralize feature flags

You already have multiple providers and features. Consolidate into a single config object loaded once, to avoid scattering environment checks across services.

```
# phase1/services/config/feature_flags.py
FEATURES = {
    "risk_desk": True,
    "risk_desk_demo_mode": True,
    "broker_live": False,
    "nova_provider": False,          # optional; enabled only when configured
    "autopilot_trade_exec": False,
}
```

1.6 Frontend Fixes (Navigation + Stability)

Fix D: LeftNavEnhanced.tsx `expanded` variable bug

In `frontend/src/features/layout/shell/LeftNavEnhanced.tsx` there is a top-level constant `expanded = true` that is referenced inside the component. This defeats state and can cause inconsistent UI.

- Replace constant with React state (`useState`) or derived layout state.
- Add an E2E smoke test that loads the app, switches views, and confirms tabs render.

Fix E: Standardize loading/error states for all views

Risk Desk will add more async flows (tool traces, streaming). Standardize a pattern: loading skeleton, empty state, recoverable error state, fatal error state. Use the same visual language across Options + Autopilot + Risk Desk.

1.7 “Platformization” Change (Critical for future hackathons)

Your explicit requirement: Nova is just a phase. Therefore, add a provider-agnostic orchestration layer now. That way, future hackathons reuse the same pipeline with different models or capabilities.

- Define `LLMProvider` interface and `LLMRouter` that selects provider (groq, gemini, nova, mock) based on env.
- All agents depend only on the interface, never directly on Bedrock/Groq/Gemini SDKs.
- Demo mode defaults to `mock` provider (no external calls), but keeps the full tool trace pipeline.

Part 2 – Hackathon-Winning Scope: Minimal Workflow, Max Credibility

This plan intentionally biases toward what judges reward: reliability, technical clarity, and a cohesive narrative. You can add breadth later; in the submission window you want a single workflow that cannot fail.

2.1 The One Workflow (Non-Negotiable)

End-to-end flow (demo mode):

- **Input:** Upload options portfolio CSV (or click “Load Demo Portfolio”).
- **Data:** Synthetic chain snapshot + synthetic underlyings (deterministic fixture).
- **Compute:** Deterministic tools compute Greeks and stress P&L.;
- **Recommend:** Agent proposes 2 hedges (rule-based + light LLM explanation).
- **Verify:** Independent verifier recomputes with alternate method and checks tolerance.
- **Gate:** Compliance engine blocks undefined-risk trades and requires user action.
- **Execute (sandbox):** Generate a trade ticket; optional UI prefill (automation) must stop before submit.

2.2 Acceptance Criteria (Judge-Proof)

Define pass/fail criteria so you can ship without debating scope daily.

Area	Must pass	How verified (during demo)
Determinism	Same inputs => same outputs	Run demo twice; hashes and P&L match.
No hallucinated numbers	All numeric outputs sourced from tools	Tool Trace panel shows JSON inputs/outputs + duration.
Verifier independence	Different algorithm produces close results	Show delta/gamma/vega/theta deviations < tolerance.
Compliance gating	Unsafe position blocks pipeline	Include naked option in demo portfolio; gate triggers.
Zero-config run	Judges can run without keys	`make demo` boots API+UI; auto-load fixture.
Failure handling	Bad CSV doesn't crash UI	Upload adversarial CSV; error displayed; pipeline halts safely.
Nova optionality	Pipeline runs without Nova	Set LLM_PROVIDER=mock; same deterministic tool outputs appear.

2.3 What NOT to Build Before Submission (Explicitly Out of Scope)

- Full historical backtesting engine (slippage, borrow, dividends, transaction cost model).
- Live broker execution, real orders, or anything requiring KYC/credentials during the demo.
- Multi-asset correlation matrix, cross-asset Greeks, portfolio optimization.
- News/sentiment/RAG-based research (distracting + fragile).
- Complex stochastic vol models (Heston calibration) and claims you can't defend live.

Part 3 – Product Specification: Options Risk Desk Module

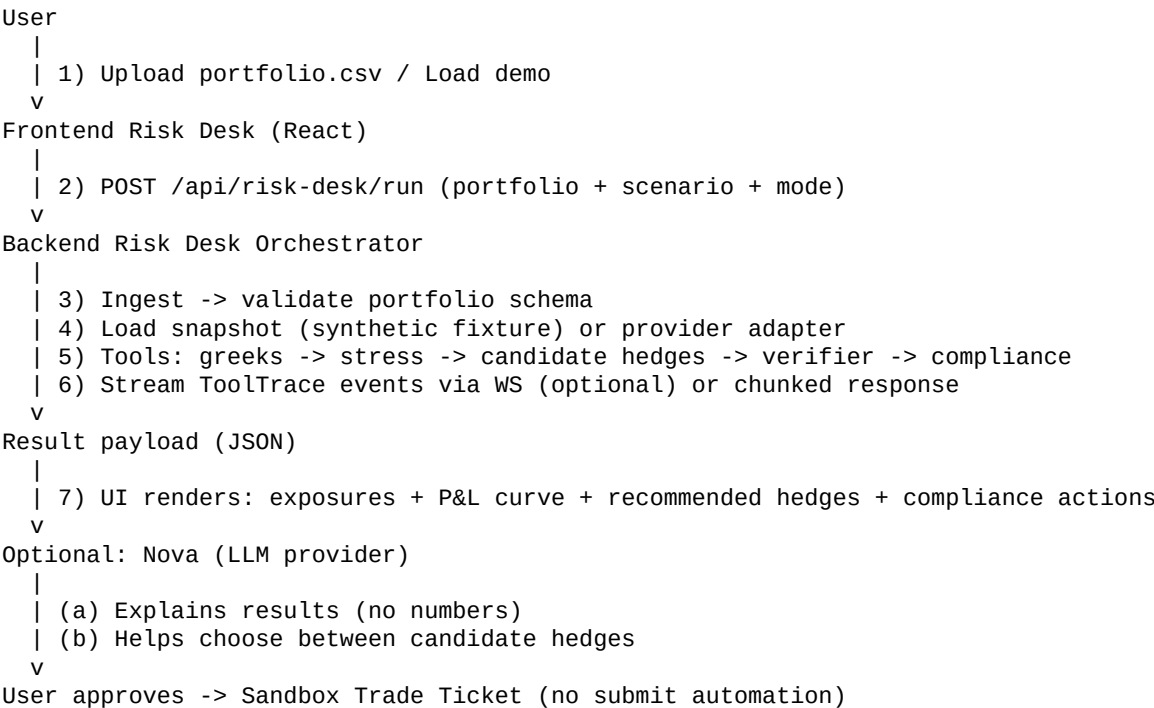
This is the complete functional spec for the hackathon feature, including contracts between backend and frontend. It is written so you can implement without further brainstorming.

3.1 Naming & Placement in Existing Repo

To minimize churn, add a new service module under `phase1/services/` and a new UI surface under the existing Options view.

Layer	New module path	Rationale
Backend service	phase1/services/risk_desk/	New business domain; avoids cluttering options/autopilot
Backend router	phase1/services/api/routers/risk_desk.py	Keeps API routing consistent with existing pattern
Shared schemas	phase1/services/risk_desk/schemas/	Pydantic models for portfolio, chains, runs, tool traces
Synthetic data	phase1/services/risk_desk/fixtures/	Demo portfolio.csv + snapshot.json + scenarios.json
Frontend feature	frontend/src/features/options/riskDesk/	Extends existing Options feature; avoids new top-level view unless
Frontend UI state	frontend/src/features/options/riskDesk/state/	Zustand/Redux (whatever you're using) isolated from rest of app

3.2 System Context & Data Flow (Text Diagram)



3.3 Domain Model (Canonical Objects)

Standardize these core objects and use them everywhere (Options, Autopilot, Risk Desk). This reduces schema drift and future hackathon pain.

3.3.1 Portfolio CSV Canonical Schema

You already proposed a leg-based CSV. Keep it. Ensure it supports: multi-leg strategies, stock legs, and easy grouping.

```
strategy_id,leg_id,symbol,type,strike,expiry,quantity,side,entry_price,contract_multiplier
IC_SPY_001,1,SPY,put,440,2026-03-20,10,short,4.80,100
...
```

Backend converts CSV to canonical JSON:

```
Portfolio {
  as_of: datetime,
  positions: [Position],
  strategies: [StrategyGroup],
  warnings: [ValidationWarning],
}
Position {
  strategy_id: str,
  leg_id: int,
  symbol: str,
  asset_type: "option"|"stock",
  option_type?: "call"|"put",
  strike?: float,
  expiry?: date,
  qty: int,
  side: "long"|"short",
  entry_price: float,
  multiplier: int,
}
```

3.3.2 Chain Snapshot Schema (Synthetic or BYO Provider)

The snapshot is a provider-agnostic container. It must store only what tools need, no vendor-specific contracts/tickers in the public repo.

```
Snapshot {
  metadata: { snapshot_id, schema_version, notice, generated_at, content_hash },
  underlying: { SYMBOL: { spot, iv_30d?, ts? } },
  chains: { "SYMBOL_YYYY-MM-DD": { "440P": Quote, "450C": Quote, ... } }
}
Quote { bid, ask, last, iv, volume?, open_interest?, ts? }
```

3.3.3 Risk Run & Audit Log

Every run emits an audit trail. In demo mode it is stored in memory and downloadable as JSON. In dev/prod it can be stored in Postgres.

```
RiskRun {
  run_id: str,
  created_at: datetime,
  mode: "demo"|"dev"|"prod",
  portfolio_hash: str,
  snapshot_hash: str,
  scenario: ScenarioSpec,
  results: RiskResults,
  tool_trace: [ToolTraceEvent],
  compliance: ComplianceResult,
}
ToolTraceEvent {
  idx: int,
  tool_name: str,
  started_at: datetime,
  duration_ms: int,
  input_json: object,
```

```
output_json: object,  
status: "ok"|"error",  
error?: { code, message }  
}
```


Part 3 (continued) – Deterministic Tools, Agents, and Provider-Agnostic AI

3.4 Deterministic Tool Suite (Source of Truth for Numbers)

A hard rule: all numeric outputs shown in UI come from deterministic tools. LLMs (Nova/Groq/Gemini) only provide explanations, selection among already-computed candidates, and error messaging.

Tool T1: portfolio_validator

Validates CSV schema + logical consistency (side/quantity, multiplier conventions, expiry dates).

Tool T2: greeks_calculator (primary)

Closed-form Black–Scholes Greeks for European options. Inputs: spot, strike, IV, rate, DTE. Output: per-leg Greeks and aggregated portfolio exposures.

Tool T3: stress_tester

Applies scenario shock to spot/IV and computes (a) new option values, (b) P&L; by leg/strategy/portfolio, (c) max loss estimate for defined-risk spreads/condors.

Tool T4: hedge_generator

Generates 2–3 candidate hedges from a small library (put spread, collar, call spread, delta-hedge). Uses heuristics tied to exposures and constraints.

Tool T5: greeks_verifier (independent)

Binomial tree pricing + finite-difference Greeks. Should differ slightly from B–S and pass within tolerance.

Tool T6: compliance_checker

Rules engine; blocks undefined risk (naked options) and requires user action.

Tool T7: ticket_builder

Converts approved hedge into a sandbox trade ticket payload consumable by frontend.

Hackathon minimal set: T1, T2, T3, T5, T6, T7. Hedge generation can be part of T3 initially if you want fewer tools.

3.5 Agent Orchestration (Keep it Simple)

Do not over-index on agent count. You can implement as either: (a) 4 explicit agents, or (b) one orchestrator calling tools in sequence with ‘agent labels’ for UI. Judges care about behavior, not number of classes.

Logical agent	Responsibility	Depends on Nova	Output
Ingest	Parse + validate portfolio	No	Canonical Portfolio + validation warnings
Risk Engine	Run T2/T3, compute exposures + P&L	Optional (explanations)	Risk Results + tool trace
Verifier	Run T5, compare with primary	No	VerificationResult
Compliance	Run T6 and block if needed	No	ComplianceResult + required actions

3.6 Provider-Agnostic LLM Layer (Nova is a Plug■in)

You already have Groq/Gemini and `LLM\_MODE=hybrid`. Extend the existing provider abstraction; do not fork the app into 'Nova-only'.

3.6.1 Interface

```
class LLMPProvider(Protocol):
    name: str
    def chat(self, messages: list[dict], *, json_schema: dict|None=None) -> dict:
        """Return: {text, structured_json?, usage?, raw?}"""
```

3.6.2 Router

```
# phase1/services/llm/router.py
# env: LLM_PROVIDER=mock|groq|gemini|nova
# env: LLM_MODE=off|explain|hybrid (keep your existing semantics)
```

3.6.3 Nova provider implementation (Bedrock)

Implement but keep disabled by default. The Risk Desk runs with `mock` provider for demo determinism.

```
# phase1/services/llm/providers/bedrock_nova.py
# reads AWS_REGION, AWS_PROFILE/credentials, NOVA_MODEL_ID (default us.amazon.nova-2-lite-v1:0)
# never required for demo mode
```

3.7 Nova Act / Automation (Optional, Must Be Safe)

You already have an 'AI options autopilot'. Do not allow Nova Act to submit trades. The only safe hackathon usage is: prefill fields in a sandbox trade ticket, take screenshot, stop.

- Automation only targets **your** sandbox UI route (e.g., /options/sandbox-ticket).
- Automation returns 'filled_fields' + screenshot + verification status.
- User must click Approve; the frontend submits; automation never clicks Submit.
- If automation fails, demo continues (ticket builder still provides payload; user can copy/paste).

Part 4 – UI/UX Specification (Hackathon-Grade + Reusable Platform)

This section is intentionally detailed. It defines the Risk Desk UI so you can implement quickly without redesigning mid-build. It also ensures the feature can be reused for future hackathons by swapping providers and adding workflows.

4.1 UI Integration Strategy (What to Keep From the Current UI)

Do not replace your existing shell/navigation. Instead:

- Keep `Shell.tsx` and the existing layout system (LeftNav + main content + right panels if present).
- Keep `OptionsView.tsx` as the entry point for all options-related workflows.
- Add a new tab inside OptionsView: **Risk Desk** (beside Chain / Builder / etc).
- Optionally add a left-nav shortcut to the same view (but keep one underlying route).

Rationale: your app already feels like a terminal. Judges reward continuity and polish; a new isolated UI tends to look like a hacky add-on.

4.2 Information Architecture (IA)

Proposed OptionsView tab model:

Options Tab	Existing?	Purpose	Hackathon relevance
Chain	Yes	Browse strikes, quotes, IV	Secondary; keep stable
Strategy Builder	Yes/Partial	Build multi-leg trades	Optional; can remain unchanged
Risk Desk (NEW)	No	Upload portfolio -> risk -> hedges -> compliance	Primary demo surface
Sandbox Ticket	Partial	Preview trades / paper trading integration	Secondary; used for Nova Act prefill

4.3 Risk Desk Screen: Layout Blueprint (Desktop)

The Risk Desk is a single screen composed of four regions. This keeps the demo flow linear and prevents tab-switching during the 3-minute video.

+-----+ Top Bar: [Load Demo] [Upload CSV] [Run] Mode: (Synthetic) Provider: (Mock/Nova) +-----+		
Left Column (30%)	Center (45%)	Right (25%)
-----	-----	-----
A. Portfolio Summary	C. Risk Results	E. Agent/Tool
- positions table	- Portfolio Greeks cards	Trace
- validation warnings	- Scenario selector	- stepper
- compliance status badge	- Stress P&L chart	- JSON tool I/O
	- Hedge comparison table	- durations
B. Compliance Gate Panel	D. Hedge Details	F. Audit Export
- violations list	- selected hedge payoff	- download JSON
- required user actions	- ticket preview	- run metadata
-----	-----	-----
+-----+		

Mobile is not required for hackathon. Provide a responsive fallback: stack columns, hide tool JSON behind a disclosure.

4.4 Visual Design System (Reuse Existing Theme)

Your current UI is dark-terminal oriented. Keep it. Risk Desk adds only a few semantic colors for status and safety:

Token	Meaning	Usage
--status-ok	Passed/verified	sanity checks, verifier status
--status-warn	Needs attention	validation warnings, non-blocking issues
--status-block	Blocked/unsafe	compliance gate, verifier mismatch
--accent-risk	Risk metrics emphasis	Delta/Vega cards, stress loss values
--accent-hedge	Hedge recommendation emphasis	selected hedge row, payoff overlay

Implement as CSS variables in your existing theme file; do not hardcode colors across components.

Part 4 (continued) – Frontend Component Specifications

Each component below has: purpose, data contract, UI states, and implementation notes. This is meant to eliminate design ambiguity and reduce rework.

Component: RiskDeskDashboard

Purpose: Container page; orchestrates subpanels and stores run state.

Key props/state: runState, portfolio, snapshotMeta, results, toolTrace, compliance

Implementation notes: Calls backend run endpoint; subscribes to WS trace stream; handles cancellation.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: RunControls

Purpose: Top bar controls: Load Demo, Upload, Run, Cancel, Provider toggle.

Key props/state: onLoadDemo, onUpload, onRun, onCancel, mode, provider

Implementation notes: Disable Run until portfolio valid; show spinner while running.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: PortfolioUpload

Purpose: CSV drag-drop + file picker + sample download.

Key props/state: onFile, onError

Implementation notes: Parse client-side optionally; always re-validate server-side.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)

- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: PortfolioSummaryTable

Purpose: Compact positions table grouped by strategy_id.

Key props/state: portfolio

Implementation notes: Inline badges for long/short; show warnings tooltip.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: ValidationBanner

Purpose: Displays schema errors and non-fatal warnings.

Key props/state: issues

Implementation notes: Non-fatal warnings allow Run; fatal errors block Run.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: DataProvenancePanel

Purpose: Shows snapshot ID, schema version, hash, mode; no vendor claims.

Key props/state: snapshotMeta

Implementation notes: In demo: shows 'Synthetic fixture'; provides Copy Hash button.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status

- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: ScenarioSelector

Purpose: Selects stress scenario presets or custom shocks.

Key props/state: scenario, onChange

Implementation notes: Presets: -5% spot + 1.5x IV; -10% spot + 2x IV; +5% rally + 0.8x IV.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: GreeksCards

Purpose: Portfolio-level Greeks summary cards (Delta/Gamma/Vega/Theta).

Key props/state: greeks, verificationStatus

Implementation notes: If verifier mismatch -> red border + tooltip with deviation.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: StressPnLChart

Purpose: Plots P&L; vs underlying price (and optionally IV).

Key props/state: pnlCurve, selectedHedgeCurve

Implementation notes: Reuse existing chart infra; allow hover tooltip; show max loss.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)

- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: HedgeCandidatesTable

Purpose: Shows 2-3 hedge candidates with cost and risk reduction.

Key props/state: candidates, selectedId, onSelect

Implementation notes: Row selection updates HedgeDetailPanel; show 'recommended' badge.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: HedgeDetailPanel

Purpose: Explains selected hedge and shows payoff diagram overlay.

Key props/state: hedge, analysis

Implementation notes: LLM explanation text only; numeric fields from tools.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: ComplianceGatePanel

Purpose: Shows rule violations; blocks execution until resolved.

Key props/state: compliance, onResolveAction

Implementation notes: Two action types: accept fix (adds hedge leg) or acknowledge risk (demo only).

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: TicketPreviewPanel

Purpose: Shows sandbox trade ticket JSON and editable fields.

Key props/state: ticket, onEdit, onApprove

Implementation notes: Approve triggers optional Nova Act prefill or local submit.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: ToolTracePanel

Purpose: Streaming list of tool calls with inputs/outputs; collapsible.

Key props/state: toolTrace

Implementation notes: Default collapsed JSON; expand per tool; show duration and status.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: AgentStepper

Purpose: High-level steps: Ingest -> Risk -> Verify -> Comply -> Ticket.

Key props/state: stepStatus

Implementation notes: Pure UI; status derived from tool trace milestones.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Component: AuditExportPanel

Purpose: Download run audit JSON + copy run_id.

Key props/state: run

Implementation notes: No DB required; offer 'Save locally' button.

UI states:

- Empty: no portfolio loaded
- Loading: run in progress (show skeleton + spinner, disable destructive actions)
- Success: show results + verification/compliance status
- Recoverable error: tool failure (show partial results + error badge)
- Fatal error: schema invalid or snapshot missing (block run, show fix instructions)

Part 4 (continued) – UI Flows, States, and Frontend Engineering Details

4.5 Primary Demo Flow (Step-by-Step UI Behavior)

This is the exact sequence you record in the demo video. Every step should have deterministic UI output.

Step	User action	Frontend behavior	Backend call(s)
1	Click “Risk Desk” tab	Loads RiskDeskDashboard; shows empty state	GET /api/risk-desk/status
2	Click “Load Demo”	Loads demo_portfolio.csv from fixtures; runs validate	POST /api/risk-desk/validate
3	Click “Run”	Starts run; opens WS for trace; sets stepper	POST /api/risk-desk/run + WS /ws/risk-desk/{run_id}
4	Watch results fill in	Greeks cards update; P&L chart renders; hedge events update; final run payload saved	
5	Select hedge candidate	Detail panel updates; ticket preview created	No new call (already computed)
6	Compliance blocks	Compliance panel shows violation + suggested fix	No new call
7	Apply fix	Adds hedge legs; reruns compliance check; ticket disabled	POST /api/risk-desk/resolve
8	Prefill ticket (optional)	Triggers automation; shows screenshot; awaits approval	POST /api/risk-desk/nova-act/prefill (optional)

4.6 Adversarial / Failure Flow (Must be included once)

Demonstrate that the system refuses to guess. This is a credibility multiplier.

- Upload CSV with ambiguous ticker (e.g., BRK.B) => Ingest returns ‘needs clarification’ error; UI shows resolution choices.
- Upload CSV with strike not present in snapshot => Market data step flags missing; UI proposes nearest strikes but compliance requires explicit user acceptance.
- Upload CSV with multiplier mismatch => validation warning; run still possible but requires acknowledgement.

4.7 Frontend Data Layer & State Management

Implement a dedicated Risk Desk state store to avoid leaking state into other features. If you’re using Zustand/Redux already, follow that convention.

```
RiskDeskState:
  mode: 'demo' | 'dev'
  provider: 'mock' | 'groq' | 'gemini' | 'nova'
  portfolio: Portfolio | null
  snapshotMeta: SnapshotMeta | null
  run: RiskRun | null
  toolTrace: ToolTraceEvent[]
  stepStatus: {ingest, risk, verify, comply, ticket}
  ui: {selectedHedgeId, scenarioPreset, showJsonTrace: boolean}
Actions:
  loadDemo(), uploadPortfolio(file), validate(), run(), cancel(), selectHedge(id),
  applyFix(action), exportAudit()
```

4.8 WebSocket Streaming UI Pattern

Use your existing WS infrastructure patterns. Risk Desk WS stream must be idempotent and resumable. If WS fails, fall back to polling / final payload.

```
WS event types:
  tool_started {idx, tool_name, input_hash, ts}
  tool_finished {idx, tool_name, output_json, duration_ms, ts}
  tool_failed {idx, tool_name, error, ts}
  run_completed {run_id, results_summary, ts}
```

4.9 Accessibility & Keyboard Support (Lightweight but Real)

- All buttons accessible by keyboard; tab order from RunControls -> Portfolio -> Results -> Ticket.
- ARIA labels for file upload, scenario selector, tool trace expand/collapse.
- Color is never the only indicator: use icons + text for statuses.
- Provide a reduced-motion option for animated steppers.

4.10 UX Copy (Exact Text to Avoid Risky Claims)

Avoid mentioning any vendor market data in the public demo. Use this phrasing:

```
Data mode: Synthetic fixture (public demo)
Note: Optional BYO market data adapters exist. Users must comply with their provider's terms.
Numbers: Greeks/P&L are computed by deterministic tools. LLMs provide explanations only.
```

Part 5 – Backend Specification (Endpoints, Services, Data, and Safety)

5.1 API Surface (FastAPI)

All Risk Desk endpoints must be namespaced and versionable. Keep them under ``/api/risk-desk/*`` to prevent collisions with existing routes.

Endpoint	Method	Auth	Purpose	Request/Response highlights
/api/risk-desk/status	GET	none (demo)	Returns feature flags + providers	{mode, version}
/api/risk-desk/validate	POST	none (demo)	Validate portfolio CSV/JSON	req: csv_text json; resp: {valid, issues, portfolio?}
/api/risk-desk/run	POST	none (demo)	Execute full pipeline	resp: {run_id, results, compliance, ticket, trace?}
/api/risk-desk/run/{run_id}	GET	none (demo)	Fetch run payload (fallback if RiskRun fails)	
/api/risk-desk/resolve	POST	none (demo)	Apply user resolution action (and re-calculation)	resp: updated RiskRun
/ws/risk-desk/{run_id}	WS	none (demo)	Stream ToolTrace events	event stream, resumable

5.2 Internal Service Interfaces (risk_desk/)

Define explicit interfaces so the module is testable without FastAPI.

```
risk_desk/  
  orchestrator.py      # run_pipeline(portfolio, mode, provider, scenario) -> RiskRun  
  ingestion.py         # parse_csv -> Portfolio  
  snapshot_loader.py   # load_snapshot(mode, provider?) -> Snapshot  
  tools/  
    greeks_bs.py  
    stress_test.py  
    verifier_binomial.py  
    compliance_rules.py  
    ticket_builder.py  
  audit/  
    trace_recorder.py  # append ToolTraceEvent; stream to WS if enabled  
  fixtures/  
    demo_portfolio.csv  
    demo_snapshot.json  
    scenarios.json
```

5.3 Synthetic Snapshot Generation (Demo Fixture)

Do NOT generate synthetic data on the fly during the demo run (risk of non-determinism). Pre-generate fixtures and commit them. Provide a script to regenerate them deterministically (seeded RNG).

```
scripts/generate_synthetic_snapshot.py --seed 42 --symbols SPY,AAPL,TSLA --expiry 2026-03-20 --str  
outputs:  
  phase1/services/risk_desk/fixtures/demo_snapshot.json  
  phase1/services/risk_desk/fixtures/demo_snapshot.meta.json (hashes)
```

Synthetic pricing: Black–Scholes with parametric IV surface. Include no-arbitrage sanity checks (put-call parity tolerance, convexity checks, non-negative time value).

5.4 Data Provider Adapters (Optional, Feature-Flagged)

You already have keys for Finnhub/Polygon/Tiingo. Keep those integrations but isolate them behind adapter interfaces and keep them disabled in demo mode.

```
class SnapshotProvider(Protocol):
    name: str
    def load(self, symbols: list[str], expiry: date, **kwargs) -> Snapshot:
        ...
Providers:
- SyntheticFixtureProvider (default)
- PolygonSnapshotProvider (requires key; NOT used in public demo)
- FinnhubProvider (underlyings only, if needed)
```

5.5 Compliance Rules (Deterministic)

Implement as a ruleset with uniform output format. Keep rules small and explainable.

Rule	Severity	Trigger	Example UI copy	Fix suggestion
NAKED_OPTION	block	Short call/put without covered spread	Blocked: undefined risk short option spread	Suggest converting to spread by adding long
POSITION_SIZE	warn/block	Notional or contracts exceeds defined position size	Warning: position size exceeds defined limit	Reduce quantity or acknowledge (demo)
MULTIPLIER_MISMATCH	warn	Option multiplier != 100	Warning: unusual contract multiplier	Confirm multiplier or correct CSV.
MISSING_MARKET_DATA	block	Strike missing from snapshot	Blocked: required strike not present	Select nearest strike and rerun.

Part 5 (continued) – What Happens to the AI Options Autopilot

You already have an ‘Options Autopilot’. The Risk Desk should not replace it; it should become the safety and verification layer it routes through. This makes your platform reusable for future hackathons (swap LLM, keep deterministic safety core).

5.6 Autopilot + Risk Desk Integration Architecture

Treat Autopilot as a strategy generator and execution assistant. Treat Risk Desk as the risk oracle + compliance gate + verifier. Autopilot cannot submit a strategy unless Risk Desk approves.

```
Autopilot (existing)
- generates candidate strategy / trade idea
- may call LLM for rationale
- may integrate with broker/paper trading

Risk Desk (new)
- computes exposures + stress P&L deterministically
- verifies via independent model
- applies compliance rules
- returns APPROVED | BLOCKED + required actions
```

5.7 Integration Points (Concrete)

- **Backend:** Autopilot service calls ``risk_desk.orchestrator.run_pipeline()`` as a library call (not HTTP) for speed and easier testing.
- **Frontend:** Autopilot UI shows Risk Desk approval status badge and provides a deep link to the Risk Desk run details.
- **Execution:** Only Risk Desk produces the final normalized ticket payload; Autopilot can propose but not bypass `ticket_builder`.

This turns Risk Desk into a platform primitive that future hackathons can reuse (e.g., ‘AI compliance desk’, ‘AI execution assistant’, ‘AI backtest lab’).

5.8 Autopilot Modes (Demo vs Future)

Mode	Purpose	Allowed actions	Keys required
Demo	Hackathon submission	Generate strategy idea; run Risk Desk; no sandbox ticket only	None
Dev	Local iteration	Use paper trading; optional provider adapter	Optional: Finnhub/Polygon/Tiingo
Prod-like	Future	Real broker integration (disabled until hacked)	Broker keys + auth + risk controls

5.9 Guardrails to Prevent ‘Autopilot as Trading Bot’ Concerns

- Rename user-facing language from “autopilot executes trades” to “autopilot prepares tickets”.
- Make ‘Submit’ action explicit and user-only in UI.
- Log every step and show an audit export to prove no hidden behavior.
- Include a persistent disclaimer: “Demo system. Not financial advice. Not for live trading.”

Part 6 – Execution Plan Calibrated to Your Coding Velocity

You ship fast when you have clear targets. This plan assumes ~5–6 weeks to submission. It is designed to prevent the ‘random stuff’ trap by defining daily exit criteria and a strict build order.

6.1 Velocity Assumptions (Derived From Your Repo)

Your current repo contains ~111,128 lines of application code and ~26,698 lines of tests. That implies you can comfortably produce 1–3K lines/day when focused, and you can also write tests alongside.

- Target new code for hackathon slice: 8K–12K LOC (backend+frontend combined).
- Target new tests: 1K–2K LOC (unit + smoke).
- Rule: **No new feature** until demo mode is stable and deterministic.

6.2 Branching & Work Discipline (Industrial but Lightweight)

- Create branch: `hackathon-risk-desk`.
- Create a “demo lock” script: `make demo-smoke` that boots backend+frontend and runs one full Risk Desk run headless.
- Every day ends with: (a) tests pass, (b) demo smoke passes, (c) one screenshot of Risk Desk UI state saved to `/docs/progress/`.

6.3 Day-by-Day Plan (42-Day / 6-Week Schedule)

Table includes **daily exit criteria**. If you miss a day, do not carry unfinished tasks forward unchecked; cut scope instead.

Day	Theme	Backend deliverables	Frontend deliverables	Exit criteria
1	Repo stabilization	Fix duplicated routers; add <code>/status</code> endpoint; add <code>RUN_MODE</code> + feature flags	Fix nav expanded bug; add Risk Desk tab placeholder	App boots; no TS errors; <code>/status</code> works
2	Config hardening	Centralize env parsing; implement provider registry (mock/groq/gemini placeholder)	Add <code>ProviderSelector</code> UI stub	Switch providers in UI persists state; no runtime crashes
3	Risk Desk module scaffold	Create <code>risk_desk/</code> skeleton + orchestrator stub + fixtures folder	Create <code>RiskDeskDashboard</code> skeleton layout (4 regions)	Route renders; empty state visible
4	CSV ingest v1	Implement <code>parse_csv</code> -> canonical Portfolio; validate required columns	<code>PortfolioUpload</code> component + table render	Upload demo CSV -> portfolio table shows rows
5	Schema validation	Implement <code>portfolio_validator</code> tool; return structured issues	<code>ValidationBanner</code> UI + blocking behavior	Bad CSV shows errors and disables Run
6	Synthetic fixture v1	Add <code>demo_snapshot.json</code> (small) + loader; compute content hash	<code>DataProvenancePanel</code> displays fixture metadata	Load demo snapshot successfully; hash shown

Day	Theme	Backend deliverables	Frontend deliverables	Exit criteria
7	Demo smoke harness	CLI `python -m risk_desk.demo_run` runs full pipeline stub	RunControls 'Load Demo' button wired	One command produces deterministic JSON output
8	Greeks tool (B-S)	Implement greeks_bs with unit tests	GreeksCards UI renders from mocked results	Unit tests pass; portfolio greeks show
9	Stress tool v1	Implement stress_test spot shock + IV multiplier; compute P&L;	StressPnLChart placeholder + numbers table	Scenario preset shows worst-case loss
10	Payoff curve generation	Generate P&L; curve across spot grid; deterministic	Wire chart to curve data; hover tooltip	Chart renders curve and tooltips
11	Hedge candidate generator	Heuristic hedge library + candidate scoring	HedgeCandidatesTable UI with selection	2 candidates appear with cost/risk reduction
12	Ticket builder	Normalize hedge into ticket payload JSON	TicketPreviewPanel shows payload + copy button	Ticket payload copies; no submit
13	Verifier binomial v1	Binomial pricer + finite diff Greeks; tests	Verifier status badge UI	Deviation computed and displayed
14	Compliance rules v1	Implement NAKED_OPTION + MISSING_DATA; tests	ComplianceGatePanel blocks ticket; shows fix actions	Demo portfolio triggers a block reliably
15	Orchestrator wiring	run_pipeline executes tools in order; tool trace recorder	AgentStepper lights up based on milestones	End-to-end run returns results
16	Tool trace storage	ToolTraceEvent model; in-memory store by run_id	ToolTracePanel with collapsible JSON	Trace streams/loads; expandable
17	WebSocket stream	WS endpoint streams tool events; fallback GET run_id	UI subscribes and updates progressively	Kill WS -> fallback loads final payload
18	Resolve actions	Endpoint to apply fix (convert naked option to spread)	UI action button applies fix and reruns compliance	One-click fix unblocks ticket
19	Adversarial cases	Add ambiguous ticker + missing strike cases	UI shows resolution modals	Uploading adversarial CSV doesn't crash
20	UI visual polish 1	No backend changes (bugfix only)	Spacing/hierarchy, skeletons, badges, icons	Risk Desk readable and demo-ready
21	Performance pass	Optimize tool runtime; cache snapshot; measure durations	Show durations in ToolTracePanel	Full run completes <30s
22	LLM explanation interface	Implement LLMPProvider interface; mock provider returns canned text	Show explanation panel (text only)	Demo runs with mock provider end-to-end

Day	Theme	Backend deliverables	Frontend deliverables	Exit criteria
23	Groq/Gemini adapter cleanup	Wrap existing Groq/Gemini usage in provider interface	Provider selector works with existing keys	Switch providers without code changes
24	Nova provider (optional)	Add bedrock_nova provider; disabled by default	Provider selector shows Nova only when enabled	Nova can be enabled locally without breaking demo
25	Nova in risk engine (optional)	LLM generates explanation + chooses candidate among computed hedges	UI shows 'LLM explanation' + provenance disclaimer	No numbers from LLM; tool trace confirms
26	Nova Act (optional)	Implement sandbox prefill endpoint (no submit)	Show screenshot + Approve button flow	Automation can fail gracefully
27	Autopilot integration	Autopilot calls risk_desk gate before ticket output	Autopilot UI shows 'Risk Approved' badge	Autopilot cannot bypass compliance
28	Docs & disclaimers	Write HACKATHON.md + demo runbook	Add in-app help drawer for Risk Desk	A new user can run demo from README
29	Test matrix expansion	Add unit tests for tools; property-based for parity/convexity	Add vitest smoke for Risk Desk UI	CI passes; coverage improves
30	E2E smoke	Add playwright/cypress (optional) or node script to hit endpoints	One-click demo path tested	No regressions on fresh clone
31	Error polish	Human-readable errors; no stack traces to UI	Error states polished with actions	Adversarial demo looks intentional
32	Demo script rehearsal	Freeze demo data; record timings	Finalize UI layout for recording	Run is stable and consistent
33	Video capture day	No backend changes except critical bugfix	Record 3-min video; add captions	Video done
34	Buffer / bugfix	Fix issues discovered during video capture	Minor UI tweaks	Nothing breaks
35	Submission packaging	Tag release; export audit sample; verify make demo	Add screenshots + architecture diagram	Judge can run it in <2 minutes
36	Post-submission prep	Create roadmap doc + next epics tickets	Polish for reuse in next hackathon	Backlog ready
37	Future provider adapters	Optional: clean adapter interface for new hackathons	UI flags for hiding/showing modules	Provider swaps are trivial
38	Refactor sprint	Consolidate autopilot/autopilot_v2 if needed	Deduplicate UI components	Tech debt reduced
39	Performance sprint 2	Batch computations, caching, profiling	Render optimization	Run <10s
40	UX polish sprint	Micro-interactions, keyboard, accessibility	Finalize visual language	Feels 'product' not 'prototype'

Day	Theme	Backend deliverables	Frontend deliverables	Exit criteria
41	Documentation polish	API docs, developer guide, architecture diagrams	Onboarding tour	Ready for contributors
42	Final QA	Run on clean machine; fix env issues	Visual regression check	Reusable baseline for next event

Part 7 – Testing, QA, Observability, and Secrets Management

7.1 Test Strategy (Minimum for Hackathon + Foundation for Later)

You already have extensive tests. For the Risk Desk slice, prioritize deterministic tool correctness and demo stability over broad coverage.

- **Unit:** Tools (B–S Greeks, stress test, binomial verifier, compliance rules).
- **Contract:** API request/response schemas (Pydantic) + fixture-based golden tests.
- **Frontend:** Vitest render tests for Risk Desk components + one integration test for the full run using mocked API.
- **Smoke:** ``make demo-smoke`` spins server + runs a headless pipeline and confirms hashes.

7.2 Golden Fixtures and Determinism Checks

Create goldens in ``phase1/services/risk_desk/fixtures/golden/``. Every demo build compares current outputs to goldens.

```
golden/
  run_result.json
  tool_trace.json
  pnl_curve.json
  compliance.json
  ticket.json
```

Add a script: ``scripts/verify_golden.py`` that fails if diffs exceed tolerance (e.g., Greeks deviations < 1e-6, P&L; curve within cents).

7.3 Observability (Audit First, Logs Second)

Judges trust what they can see. Your UI should show the audit trace; logs are secondary.

- ToolTraceEvent stream is the primary ‘observable’.
- Log each `run_id`, `portfolio_hash`, `snapshot_hash`, tool durations.
- Expose a ‘Download audit JSON’ button in UI.

7.4 Secrets and Environment Variables

You provided ``keys.env.example``. Keep that file. Never commit real keys. Add guardrails so the app refuses to start in prod-like mode if keys are missing, but demo mode ignores them.

```
RUN_MODE=demo
LLM_PROVIDER=mock
ENABLE_POLYGON=0
ENABLE_FINNHUB=0
ENABLE_NOVA=0
```

In dev mode, allow optional keys: `FINNHUB_API_KEY`, `POLYGON_API_KEY`, `GROQ_API_KEY`, `GEMINI_API_KEY`. In demo mode, ignore all and use fixtures.

7.5 CI Checklist (Light)

- ``pytest -q`` for tool correctness tests

- ``npm test`` or ``vitest run`` for frontend
- ``python scripts/verify_golden.py`` for deterministic outputs
- ``python -m risk_desk.demo_run`` for smoke

Part 8 – Submission Packaging (Docs, Commands, and Demo Reliability)

8.1 Required Docs to Add/Update

File	Purpose	What to include (exact)
HACKATHON.md	Judge front door	One command run; Nova usage map; data provenance; disclaimers; demo steps
README.md	Public landing	Short pitch; setup; links to HACKATHON.md; screenshots
docs/ARCHITECTURE_RISK_DESK.md	Deep dive	System diagram; modules; tool list; safety gates; verification
docs/DEMO_RUNBOOK.md	Operator script	Exact click-by-click + what to say; recovery if something fails
docs/PROVENANCE.md	Licensing defense	Synthetic-first explanation; BYO-data stance; what is/ isn't in repo

8.2 One-Command Demo (Makefile Targets)

Judges should not manually start three terminals. Provide make targets that handle everything:

```
make demo
- starts backend (demo mode)
- starts frontend
- opens browser
make demo-smoke
- runs headless pipeline once
- asserts golden hashes
```

8.3 Demo Script (3 Minutes, No Branching)

Use the exact steps below. Avoid switching tabs or explaining implementation internals unless asked.

- 0:00–0:20: show existing terminal UI; say “I’m adding Risk Desk.”
- 0:20–0:35: click Risk Desk; show Data Provenance (Synthetic fixture, hash).
- 0:35–1:30: click Load Demo → Run; show tool trace streaming; Greeks + stress P&L; render.
- 1:30–2:10: show verifier comparison (two methods, within tolerance).
- 2:10–2:35: show compliance block (naked option); apply fix; ticket enabled.
- 2:35–3:00: show sandbox ticket prefill (optional) + audit export + closing line.

8.4 Phrasing That Avoids Risky Claims

Do not claim: live trading, price prediction, backtest win-rate, proprietary data. Do claim: deterministic risk computation, verification, compliance gates, reproducible demo mode.

Part 9 – 12-Month Roadmap (Detailed, Platform-First)

This roadmap is designed so that each quarter can be a separate hackathon theme while reusing the same core: deterministic tools + audit + provider adapters.

9.1 Roadmap Principles

- Keep the deterministic tool core as the 'platform kernel'.
- Treat data providers and LLMs as replaceable plugins.
- Ship vertical slices: each quarter delivers a user-visible workflow improvement and a demo-able story.
- Avoid deep model claims; emphasize correctness, verification, and safety.

9.2 Quarter-by-Quarter Plan

Quarter	Theme	Major deliverables	Engineering focus	Success metrics
Q1 (Months 1–3)	Risk Desk v1 → v1.5	Harden demo; add richer scenarios; basic reporting PDF; improve autopilot gating	Tool correctness, UI polish, docs, test automation	<10s run time; <1 crash per 50 runs; 80%+ 'demo succeeds' rate on fresh machine
Q2 (Months 4–6)	Real data adapters + alerts	BYO Polygon/Alpaca/Tiingo adapters; rate limit/caching; real-time Greeks updates; Slack/email alerts	Provider abstraction, caching, streaming updates	Adapters stable; 95% uptime in dev; 1-click provider switch
Q3 (Months 7–9)	Backtest-lite → Backtest v1	Scenario replay library, historical surfaces import (licensed), position lifecycle tracking, strategy evaluation UI	Data model expansion, job queue, storage	Backtest runs reproducibly; 1K scenario runs < 5 min
Q4 (Months 10–12)	Productization	API product, multi-account, compliance exports, billing (optional), user auth	Access control, tenancy, observability	First paying users; automated deployments; audit export accepted by target users

9.3 Detailed Epic Breakdown (Month-by-Month)

Each month includes: product objective, backend epics, frontend epics, and risk items.

Month	Objective	Backend epics	Frontend epics	Risks / mitigations
1	Hackathon submission stability	Finalize deterministic fixtures; golden tests; tool trace streaming	Risk Desk UX polish; stable demo mode	Scope creep → enforce acceptance criteria
2	Add scenario library	Add scenario presets; export run as JSON/PDF	Scenario selector UI; report download	Overfitting to synthetic → keep generic

Month	Objective	Backend epics	Frontend epics	Risks / mitigations
3	Autopilot safety integration	Autopilot requires risk gate approval; sandbox ticket workflow	Autopilot badge + deep link to Risk Desk run	User confusion → clear copy + help drawer
4	Provider adapters v1	Polygon/Tiingo snapshot adapters; caching; retries	Provider selector; adapter status UI	Licensing → BYO only; no redistribution
5	Realtime updates	WS pushes underlying spot updates; incremental Greeks refresh	Live updating Greeks cards + throttling	Perf → batch updates; debounce rendering
6	Alerts	Slack/email alerts for risk thresholds	Alert config UI	Spam → rate limit; user thresholds
7	Scenario replay engine	Job runner; scenario replay with stored snapshots	Backtest panel in Risk Desk	Compute cost → queue + progress UI
8	Strategy evaluation	Metrics: worst-case loss, cost, drawdown; compare hedges	Comparison dashboards	Metric correctness → unit tests + documentation
9	Data retention	Persist runs/portfolios; versioning; audit export	History view with filters	Privacy → per-user tenancy + encryption
10	Auth + accounts	User login; multi-account portfolios	Account switcher UI	Security → minimal scopes; OWASP checklist
11	API product	REST API for risk runs; API keys	Developer settings UI	Rate limiting → quotas + monitoring
12	Monetization + polish	Billing integration (optional)	Pricing page + usage stats	Distraction → only after core stable

9.4 Reuse for Future Hackathons (Explicit Strategy)

Design each future hackathon entry as a new 'workflow module' reusing the same kernel:

- Kernel: deterministic tools + audit + schema + provider adapters.
- Workflow modules: Risk Desk, Compliance Desk, Backtest Lab, Execution Prep, Research Assistant.
- Provider plugins: Nova today, other LLM tomorrow. No workflow depends on a single provider.
- Data plugins: synthetic fixtures always available so demos never require external keys.

Appendix A – Detailed Retention Map (Current Repo → Future State)

This appendix lists major folders and what happens to them. Use it as a checklist to avoid accidental deletions.

Path	Keep?	Change type	Notes
frontend/	Yes	Extend	Add Risk Desk tab/components; keep shell, existing views.
phase1/	Yes	Extend	Add risk_desk service + router; minimal refactors for stability.
docs/ (or *.md at root)	Yes	Curate	Keep; add hackathon docs; move outdated notes to /docs/archive.
keys.env.example	Yes	Harden	Keep template; ensure keys.env ignored; add startup checks by mode.
vendor integrations (finnhub/polygon/tiingo/alpaca/tradier)	Yes	Feature-flag	Disabled in demo; used in dev only.

Appendix A.1 phase1/services/ Directory Inventory

Service	Primary responsibility	Risk Desk interaction	Keep/Refactor notes
alerts	Existing module	None (indirect)	Review:
api	Existing module	Hosts Risk Desk routes + WS	Refactor: Router hygiene, remove duplicates, add /risk-desk endpoints + WS
automation	Existing module	None (indirect)	Review:
autopilot	Existing module	Consumes Risk Desk compliance gate outputs; cannot bypass ticket builder	Integrate: Autopilot becomes a consumer of risk desk gates; keep strategy logic, add pre-trade validation
backtest	Existing module	None (indirect)	Review:
bar_engine	Existing module	None (indirect)	Review:
charting	Existing module	None (indirect)	As-is: Same as above
clock	Existing module	None (indirect)	Review:
delivery	Existing module	None (indirect)	Review:
execution	Existing module	None (indirect)	Review:
forecasting	Existing module	None (indirect)	Review:
fundamentals	Existing module	None (indirect)	Review:
incidents	Existing module	None (indirect)	Review:
ingestion	Existing module	None (indirect)	Review:

Service	Primary responsibility	Risk Desk interaction	Keep/Refactor notes
llm	Existing module	None (indirect)	Extend: Add Bedrock/Nova provider without forcing Nova
monitoring	Existing module	None (indirect)	Review:
options	Existing module	Shares option pricing and chain data structures; Risk Desk extends calculators	Extend: Add portfolio/greeks/pnl calculators or reuse if present
packages	Existing module	None (indirect)	Review:
parity	Existing module	None (indirect)	Review:
persistence	Existing module	None (indirect)	Review:
portfolio	Existing module	Canonical portfolio schema; used by both Autopilot and Risk Desk	Refactor: Unify schemas; ensure options legs can be expressed cleanly
recovery	Existing module	None (indirect)	Review:
replay	Existing module	None (indirect)	Extend: Deterministic snapshots; risk desk piggybacks replay infra
reports	Existing module	None (indirect)	Review:
strategy	Existing module	None (indirect)	Review:
tts	Existing module	None (indirect)	Review:
verifier	Existing module	None (indirect)	Review:

Appendix A.2 frontend/src/features/ Inventory

Feature	What it is today	Risk Desk impact	Keep/Refactor notes
alerts	Existing feature	Minimal	Review:
audit	Existing feature	Minimal	Review:
autopilot	Existing feature	Show risk approval badges + deep link into Risk Desk	Integrate: Autopilot should consume compliance gate outputs
backtest	Existing feature	Minimal	Optional: Use for scenario comparisons; avoid full build in hackathon
chart	Existing feature	Minimal	Review:
dashboard	Existing feature	Minimal	As-is: Add risk summary tile later
drawings	Existing feature	Minimal	Review:
fundamentals	Existing feature	Minimal	Review:
health	Existing feature	Minimal	Review:
ide	Existing feature	Minimal	Review:

Feature	What it is today	Risk Desk impact	Keep/Refactor notes
incidents	Existing feature	Minimal	Review:
indicators	Existing feature	Minimal	Review:
layout	Existing feature	Nav route addition and shared loading/error patterns	Extend: Add Risk Desk view/tab; ensure nav remains coherent
notes	Existing feature	Minimal	Review:
observability	Existing feature	Minimal	Review:
options	Existing feature	Add riskDesk/ subfeature, new tab and components	Extend: Add Risk Desk tab + new panels
orders	Existing feature	Minimal	Review:
packages	Existing feature	Minimal	Review:
portfolio	Existing feature	Minimal	Review:
replay	Existing feature	Minimal	Review:
reports	Existing feature	Minimal	Review:
runs	Existing feature	Minimal	Review:
strategy	Existing feature	Minimal	Review:
trades	Existing feature	Minimal	Review:
trading	Existing feature	Minimal	Review:
tts	Existing feature	Minimal	Review:

Appendix B – Environment Variable Reference (Demo vs Dev vs Prod-like)

You already maintain `keys.env.example`. This appendix defines required/optional variables per mode.

Variable	Demo	Dev	Prod-like	Notes
RUN_MODE	required	required	required	demo uses fixtures; dev uses adapters
LLM_PROVIDER	mock (default)	mock/groq/ge mini/nova	enterprise choice	provider selected by router
LLM_MODE	off/explain	hybrid	hybrid	keep your existing semantics
POLYGON_API_KEY	ignored	optional	optional	BYO only
FINNHUB_API_KEY	ignored	optional	optional	BYO only
TIINGO_API_KEY	ignored	optional	optional	BYO only
GROQ_API_KEY	ignored	optional	optional	explanations
GEMINI_API_KEY	ignored	optional	optional	explanations
APCA_API_KEY_ID / SECRET	ignored	optional	optional	sandbox only until hardened
TRADIER_BROKERAGE_KEY	ignored	optional	optional	sandbox only until hardened
POSTGRES_*	ignored	optional	required	demo must not require DB
ENABLE_NOVA	0	0/1	1	gate provider availability

Appendix C – API Payload Examples (Copy/Paste)

C.1 Validate Request/Response

```
POST /api/risk-desk/validate
{
  "csv_text": "strategy_id, leg_id, symbol, type, strike, expiry, quantity, side, entry_price, contract_mul"
}
Response:
{
  "valid": true,
  "issues": [],
  "portfolio": {...}
}
```

C.2 Run Request/Response

```
POST /api/risk-desk/run
{
  "mode": "demo",
  "provider": "mock",
  "scenario": {"preset": "SPY_-5_IVx1.5"},
  "portfolio": {...}
}
Response:
{
  "run_id": "RR_20260206_0001",
  "results": {"greeks": {...}, "stress": {...}, "pnl_curve": [...]},
  "candidates": [...],
  "verification": {...},
  "compliance": {...},
  "ticket": {...},
  "tool_trace": [...]
}
```

C.3 Resolve Action Example

```
POST /api/risk-desk/resolve
{
  "run_id": "RR_20260206_0001",
  "action": {"type": "convert_to_spread", "target_leg_id": 3, "protective_strike": 195}
}
```

Appendix D – Final Pre-Submission Checklists

D.1 Demo Reliability Checklist

- Fresh clone → `make demo` works without keys.
- Risk Desk: Load Demo → Run completes twice with identical hashes.
- Adversarial CSV shows safe failure (no crash).
- Compliance block triggers and fix unblocks.
- Audit export downloads and matches UI state.

D.2 Judge-Facing Checklist

- HACKATHON.md has one-command run, screenshots, and Nova usage map.
- Video under time limit; includes failure handling once.
- README links to docs; no broken images.
- No secrets committed; keys.env ignored; repo passes secret scan locally.

D.3 Future Hackathon Reuse Checklist

- Risk Desk run_pipeline can be called as a library (no HTTP required).
- LLM providers are plugins; mock provider always available.
- Synthetic fixtures are committed and can be regenerated deterministically.
- Feature flags can hide/show modules per hackathon narrative.

Appendix E – UI Wireframes and Interaction Specifications (Implementation-Level)

This appendix provides implementation-level layout specs so you can build without Figma. Measurements are approximate; match your existing UI density.

E.1 Risk Desk Top Bar (RunControls)

Layout:

- height: 44px (align with existing header controls)
- left group: [Load Demo] [Upload CSV] [Run] [Cancel]
- right group: Mode pill, Provider dropdown, Export button

Behavior:

- Run disabled until `portfolio.valid == true` and snapshot loaded
- Cancel appears only when `runState == 'running'`
- Provider dropdown options filtered by enabled providers (feature flags + keys present)

E.2 Left Column Panels

Portfolio Summary Panel

Components:

- Header: "Portfolio" + [Download CSV] (optional)
- Table:
 - columns: Strategy | Symbol | Exp | Strike | Type | Qty | Side | Entry | IV (optional) | Δ |
- Grouping:
 - rows grouped by `strategy_id` with collapsible sections
- Row selection:
 - highlights leg; syncs with payoff chart hover

Empty state:

- show sample CSV download + "Load Demo"

Compliance Gate Panel

Hierarchy:

- Badge: APPROVED / WARNING / BLOCKED
- Violations list:
 - each violation card: Rule name, severity chip, message, 'Why it matters' expander
- Action area:
 - If BLOCKED: show 1-2 recommended actions (buttons)
 - If WARNING: show 'Acknowledge' checkbox (demo mode only)

E.3 Center Column Panels

Greeks Summary Cards

Cards (4):

Delta | Gamma | Vega | Theta

Card content:

- Big number (rounded)
- small 'per 1% move' hint (optional)
- trend indicator vs previous run (optional)

Status integration:

- if verifier mismatch: red outline + tooltip "Verifier deviation 6.2% (>5% tolerance)"

Stress P&L; Chart

Chart spec:

- X axis: underlying spot (`spot * [0.85..1.10]`)
- Y axis: P&L (\$)

- Series 1: Unhedged portfolio
 - Series 2: Hedged portfolio (when candidate selected)
 - Tooltip: spot, pnl_unhedged, pnl_hedged, delta estimate at that point (optional)
- Annotations:
- vertical line at current spot
 - label worst-case P&L in scenario window

Hedge Candidates Table

Columns:

Hedge | Cost | Max loss reduction | Worst-case (seeded) | Liquidity proxy | Notes

Sorting:

Default: recommended_score desc

Selection:

clicking row -> selects hedge and updates detail + ticket panels

E.4 Right Column Panels

Tool Trace Panel

Default view:

- list of tool calls with status icon and duration
- clicking expands JSON input/output
- show 'copy JSON' button and 'pin' to keep expanded

Performance:

- render virtualized list if >200 events (future)

Audit Export Panel

Buttons:

- Download audit JSON (RiskRun)
- Copy run_id
- Copy portfolio_hash + snapshot_hash

Appendix F – Tool Math, Edge Cases, and Tolerances

This appendix documents exactly what the tools compute, the assumptions, and the edge-case handling. This makes your demo defensible when judges ask technical questions.

F.1 Black–Scholes Assumptions (Primary Greeks Tool)

- European exercise; no dividends unless explicitly provided (demo mode: dividend=0).
- Constant volatility over the life of the option (IV from snapshot).
- Risk-free rate fixed (demo default: 0.03) or configurable.
- Contract multiplier default 100; read from CSV if provided.

F.2 Formula Reference (Used Internally; Not Shown to Users)

```
Let:
  S = spot, K = strike, T = time to expiry (years), r = rate, sigma = IV
  d1 = (ln(S/K) + (r + 0.5*sigma^2)*T) / (sigma*sqrt(T))
  d2 = d1 - sigma*sqrt(T)

Call price: S*N(d1) - K*e^{-rT}*N(d2)
Put price:  K*e^{-rT}*N(-d2) - S*N(-d1)

Delta_call = N(d1)
Delta_put  = N(d1) - 1
Gamma      = n(d1) / (S*sigma*sqrt(T))
Vega       = S*n(d1)*sqrt(T)
Theta_call = ...
Theta_put  = ...
```

Implementation note: avoid catastrophic cancellation near expiry; clamp T to min epsilon (e.g., 1e-6 years).

F.3 Binomial Verifier (American Approximation)

- Use Cox–Ross–Rubinstein (CRR) tree with N steps (demo: N=200) for stable results.
- For American options: allow early exercise at each node for puts/calls as applicable (demo can treat as American for verifier).
- Compute Greeks by finite differences: bump S by +/- dS; bump sigma by +/- dSigma; recompute prices.

Tolerance policy:

Metric	Tolerance	Rationale
Delta	<= 5% relative or <= 0.02 absolute	Avoid false positives at low deltas
Gamma	<= 10% relative	Gamma is noisy numerically
Vega	<= 7% relative	Stable enough but still numeric noise
Theta	<= 10% relative	Sensitive to time discretization

F.4 Stress Test Edge Cases

- If IV floor triggered (e.g., IV < 0.05), log a warning but continue.

- If quote missing: compliance blocks run; do not interpolate silently unless user accepts.
- If leg expiry already passed: validation blocks run.
- If portfolio contains both stock + options: include stock $\Delta = \text{qty}$, $\Gamma = 0$, $\text{vega} = 0$, $\text{theta} = 0$.

F.5 Seeded Scenario Sweep (Optional)

If you include a Monte Carlo sweep, keep it seeded and report it as 'scenario sweep' not 'backtest'.

```
seed=42
spot_shock ~ Uniform(-0.10, +0.05)
iv_mult    ~ Uniform(0.8, 2.0)
N=200
Outputs:
  worst_case_unhedged, worst_case_hedged, avg_loss_reduction
```

Appendix G – Expanded Backlog (Post-Hackathon Task Bank)

This backlog is organized to support rapid iteration and future hackathon pivots. Use it as your 'idea bank' without polluting the submission scope.

Data & Providers

- Add Alpaca options chain adapter (BYO key).
- Add Tradier chain adapter (BYO key).
- Implement unified rate-limit + cache layer for all providers.
- Add snapshot versioning and diff tools.

Risk & Analytics

- Add margin approximation per broker (heuristic).
- Add portfolio VaR (parametric) using synthetic covariance for demo.
- Add surface sanity checks (calendar/vertical arbitrage).
- Add strategy classification (condor, butterfly, ratio, calendar).

UX

- Add saved portfolios and run history timeline.
- Add compare two runs overlay (before/after hedge).
- Add inline payoff diagram per strategy group.
- Add 'Explain this metric' hover glossary.

Autopilot

- Add autopilot 'what-if' sandbox: propose 3 tickets, run Risk Desk on each.
- Add autopilot guardrails: max daily trades, max loss limits, user risk profile.
- Add broker simulation for fills (paper).

Infra

- Add job queue for heavy computations.
- Add structured logging to OpenTelemetry exporter (future).
- Add permission model (roles: viewer/trader/admin).